

1/8/2026

Jerold Samuel.J

192571052

C8A1407

①

COMPILER DESIGN

1) In given question, the grammar is

$$E \rightarrow E + E / id.$$

In this grammar which consists of two production

$$1) E \rightarrow E + E$$

$$2) E \rightarrow id.$$

But here $E \rightarrow E + E / id$ is not a ambiguous because it can generate more than one production.

41

100

4) Eliminate left recursion

$$\text{grammar : } A \rightarrow AX / B$$

$$A \rightarrow AX$$

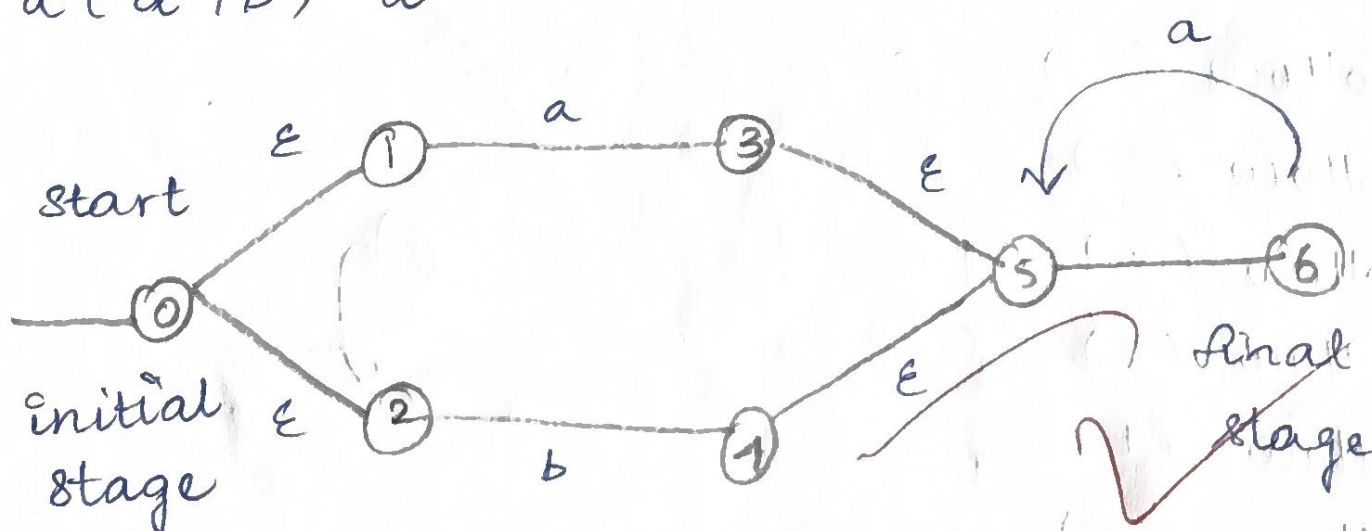
$$A \rightarrow B$$

$$A \rightarrow BA'$$

$$A' \rightarrow XA' / \epsilon$$

2)

$$a) a(a|b)^*a$$



5)

prefix : $s \rightarrow iEtS / iEtSes / a$

This grammar is found to be ambiguous and suitable for predictive parsing.

where

$s \rightarrow \underline{iEtS} / \underline{iEtSes} / a$

$s \rightarrow iEtS / a$

$s' \rightarrow es / \epsilon$

$s \rightarrow iEtSs' / a$

$s' \rightarrow es / \epsilon$

6)

$s \rightarrow aBDh$

$B \rightarrow cC$

$C \rightarrow bc / \epsilon$

$D \rightarrow EF$

$E \rightarrow g / \epsilon$

$F \rightarrow f / \epsilon$

First (S) = { a, h }

First (B) = { c }

First (C) = { b, ϵ }

First (D) = { g, ϵ }

First (E) = { g, ϵ }

First (F) = { f, ϵ }

Follow (S) = { \$ }

Follow (B) = { g, f, h }

Follow (C) = { g, f, h }

Follow (D) = { h }

Follow (E) = { f, h }

Follow (F) = { g, h }

②

given grammar: $S \rightarrow Sa / Sb / cd$.

$$S \rightarrow Sa$$

$$S \rightarrow Sb$$

$$S \rightarrow cd$$

These are three production are given by S.
Here first two steps left recursion are occurs.

$$S \rightarrow Sa$$

$$S \rightarrow Sb$$

In this two steps left recursion are occurred.

In $S \rightarrow cd$ the left recursion are not occurred because it start the letter from small letter.

8)

$$E \rightarrow TE'$$

$$E' \rightarrow TE' / \epsilon$$

Find first $(E') = ?$

$$\text{first}(E') = \{+, \epsilon\}$$

9) given grammar: $S \rightarrow (S) S / \epsilon$

string (()) string.

this is valid.

2) shift reducing parsing is technique which used in the parsing method. So that method shift reducing parsing are used in stack implementation. So that stack implementation that shift reducing parsing are included four steps. These are called actions.

(i) Shift	(iii) Accept
(ii) Reduce	(iv) Error.

3) Grammar: $S' \rightarrow \epsilon$
 $\epsilon \rightarrow CC$
 $C \rightarrow cC/d$

closure : $S' \rightarrow \cdot S$

1) $S \rightarrow AB$ $\text{First}(S) = \{a, \epsilon\}$
 $A \rightarrow a$ $\text{First}(A) = \{a, \epsilon\}$
 $B \rightarrow b$ $\text{First}(B) = \{b, \epsilon\}$

Grammar : $S \rightarrow aSb$

aaabbb

aaabbb

$\therefore S \rightarrow AB \quad A \rightarrow a$



$S \rightarrow A$

$A \rightarrow aB / Ad.$

$B \rightarrow b$

$C \rightarrow g.$

$\text{First}(S) = \{a\}$

$\text{First}(A) = \{a, d\}$

$\text{First}(B) = \{b\}$

$\text{First}(C) = \{g\}$

$\text{Follow}(S) = \{\$ \}$

$\text{Follow}(A) = \{d, \$ \}$

$\text{Follow}(B) = \{\$ \}$

$\text{Follow}(C) = \{\$ \}$

15)

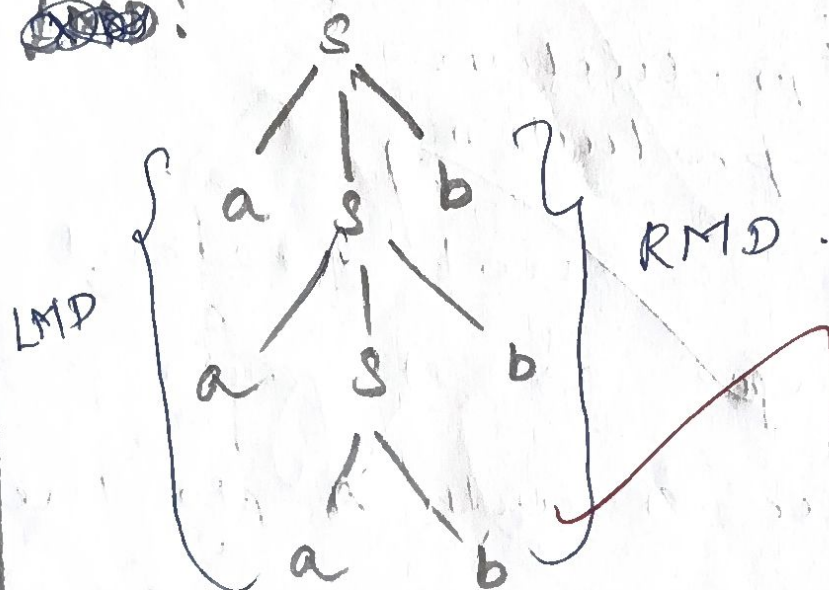
$S \rightarrow AB$

$A \rightarrow aA / a$

$B \rightarrow bB / b$

input = aaabbb.

~~LRD~~:



16)

$$S \rightarrow AB$$

$$A \rightarrow aA / \epsilon$$

$$B \rightarrow bB / c$$

$$\text{First}(S) = \{a, b, \epsilon\}$$

$$\text{First}(A) = \{a, \epsilon\}$$

$$\text{First}(B) = \{b, c\}$$

$$\text{Follow}(S) = \{ \$ \}$$

$$\text{Follow}(A) = \{b, c\}$$

$$\text{Follow}(B) = \{ \$ \}$$

17) Eliminate Left Recursion:~~XXXXXXXXXX~~

$$A \rightarrow A B \beta$$

~~A grammar is~~

(i) First identify the production it has some production.

eg: $A \rightarrow A \alpha / \beta$

$A \rightarrow A \alpha \rightarrow$ 1st production.

$A \rightarrow \beta \rightarrow$ 2nd production.

(ii) Then the A is not present the make changes as A' with β

(iii) Then replace A with A' to the given production.

eg: $A \rightarrow A \alpha / \beta$

$$A \rightarrow A \alpha$$

$$A \rightarrow \beta$$

$$A \rightarrow \beta A'$$

$$A' \rightarrow \alpha A' / \epsilon$$

32

3)

id + id * id

$$E \rightarrow TE'$$

$$E' \rightarrow +TE' / \epsilon$$

$$T \rightarrow FT'$$

$$T' \rightarrow *FT / \epsilon$$

$$F \rightarrow (E) / \underline{id}$$

$$F \rightarrow id$$

~~Q~~

$$\text{Follow}(E) = \{ \$,) \}$$

$$\text{Follow}(E') = \{ \$,) \}$$

$$\text{Follow}(T) = \{ +, \$,) \}$$

$$\text{Follow}(T') = \{ +, \$,) \}$$

$$\text{Follow}(F) = \{ *, +, \$,) \}$$

$$\text{First}(E) = \{ (, id \}$$

$$\text{First}(E') = \{ +, \epsilon \}$$

$$\text{First}(T) = \{ (, id \}$$

$$\text{First}(T') = \{ *, \epsilon \}$$

$$\text{First}(F) = \{ (, id \}$$

$$E \rightarrow TE'$$

$$E' \rightarrow +TE' / \epsilon$$

$$T \rightarrow FT'$$

$$T' \rightarrow *FT / \epsilon$$

$$F \rightarrow (E) / id$$

Non-terminal	id	+	*	(	)	\$
E	$E \rightarrow TE'$			$E \rightarrow TE'$		
E'		$E' \rightarrow +TE'$			$E' \rightarrow \epsilon$	$E' \rightarrow \epsilon$
T	$T \rightarrow FT'$			$T \rightarrow FT'$		
T'			$T' \rightarrow *FT$		$T' \rightarrow \epsilon$	$T' \rightarrow \epsilon$
F	$F \rightarrow (E) / id$			$F \rightarrow (E) / id$		

stack	input	output
\$	id + id * id \$	—
\$ E' T	id + id * id \$	$E' \rightarrow T E'$
\$ E' T' F	id + id * id \$	$T \rightarrow F T'$
\$ E' T' id	id + id * id \$	$F \rightarrow id$
\$ E' T'	+ id * id \$	$T' \rightarrow \epsilon$
\$ E'	+ id * id \$	$E' \rightarrow T E'$
\$ E' T	+ id * id \$	$T \rightarrow + T E$
\$ E' T' F	id * id \$	$T \rightarrow F T'$
\$ E' T' id	id * id \$	$F \rightarrow id$
\$ E' T'	* id \$	$E' \rightarrow \epsilon$
\$ T' F	* id \$	$T \rightarrow * F T$
\$ T' id	id \$	$F \rightarrow id$
\$	\$	$T' \rightarrow \epsilon$

19)

$$E \rightarrow E + E$$

$$E \rightarrow E * E$$

$$E \rightarrow (E)$$

Input string:

id + id * id.